DOC – GENIE

By

Prasanna J (23083301802112022)

Prathiksha P (23083301802112023)

Ramya M (23083301802112024)

Reshma M (23083301802112025)

of

ARUNACHALA ARTS AND SCIENCE (WOMEN) COLLEGE

Submitted to the

In partial fulfilment of the requirement

for the award of the degree of

BACHELOR OF SCIENCE IN ARTIFICIAL INTELLIGENCE

MANONMANIAM SUNDARANAR UNIVERSITY

TIRUNELVELI – 627012

MARCH -2026

Doc-Genie – Python Function Docstring Generator

1. Introduction

Doc-Genie v2.0 is an advanced developer utility designed to automate the generation of professional Python docstrings. Unlike basic template fillers, this tool performs Static Code Analysis using the Abstract Syntax Tree (AST) module to interpret the internal logic of a function—identifying loops, conditional branches, and mathematical operations.

By leveraging a high-performance Gradio web interface, Doc-Genie allows developers to instantly transform raw code into documented, industry-standard formats like Google and NumPy styles. This project bridges the gap between rapid development and maintainable code, providing integrated export features like PDF generation and social sharing for collaborative environments.

2. Project Description

Doc-Genie v2.0 is an intelligent static analysis utility designed to bridge the gap between rapid coding and high-quality documentation. By utilizing Python's Abstract Syntax Tree (AST), the system performs a deep scan of function structures to identify not only parameters and return types but also the internal operational logic—such as recursive calls, conditional branching, and mathematical computations.

The tool features a high-performance Gradio-based web interface that supports both direct code input and `.py` file uploads. Once analyzed, the system generates industry-standard docstrings in either Google or NumPy formats. To support professional workflows, Doc-Genie includes a robust reporting engine powered by ReportLab, allowing users to export their documented codebase as stylized PDF reports or raw TXT files, complete with timestamps and logic analysis summaries.

3. Objectives

The primary goal of Doc-Genie v2.0 is to automate the technical overhead of software maintenance. The specific objectives of this system include:

- Autonomous Documentation Synthesis: To instantly generate high-accuracy docstrings for Python functions by analyzing their logical structure, reducing manual effort by 100%.

- Standardization of Style: To enforce industry-standard documentation formats (Google and NumPy) across a codebase, ensuring consistency even in large-scale or multi-developer projects.
- Enhanced Logic Transparency: To provide developers with an immediate "plain-English" summary of a function’s behavior—including loops, conditions, and math—making complex legacy code easier to debug and understand.
- Comprehensive Knowledge Export: To offer versatile reporting tools, such as stylized PDF reports and TXT summaries, allowing teams to share documentation outside of the code editor.
- Quality Assurance in Readability: To bridge the gap between "working code" and "readable code," significantly improving the maintainability and professional quality of software projects.

4. Applications

Doc-Genie v2.0 is designed to be a versatile utility across various stages of the software development lifecycle (SDLC). Its core applications include:

- Legacy Code Restoration: One of the biggest challenges for developers is maintaining "legacy" code—old functions written without comments. Doc-Genie can instantly scan these files, identify their logic, and generate documentation to make them understandable for modern teams.
- Rapid API Documentation: For developers building web services or microservices, Doc-Genie provides an immediate way to document endpoints, parameters, and return types, ensuring that internal APIs are ready for integration.
- Standardized Team Collaboration: In a team environment, different developers often use different commenting styles. Doc-Genie acts as a "style enforcer," ensuring that every function across a shared repository follows a consistent Google or NumPy standard.
- Educational Support for Students: For beginners learning Python, Doc-Genie serves as a "Reverse-Engineer" tool. By generating a plain-English description of a function's loops and math, it helps students understand how complex code logic translates into human language.
- Code Review & Quality Assurance (QA): During the code review process, reviewers can use Doc-Genie to verify if a function’s logic matches its intended purpose. The PDF export feature serves as a professional audit trail for quality control.

5. System Architecture

The architecture of Doc-Genie v2.0 is designed for modularity and high-speed static analysis. It follows a layered approach where the frontend, logic engine, and export modules interact seamlessly.

Technology Stack

- Language Environment: Developed in Python 3.8+, ensuring compatibility with modern library features and type hinting support.
- Static Analysis Engine (AST): The core intelligence of the system. Instead of executing code (which is risky), it uses the Abstract Syntax Tree (AST) module to parse code into a structural map. This allows the system to "walk" through the logic and identify function names, arguments, loops, and operations safely.
- Frontend Framework (Gradio): Provides a high-performance, responsive web interface. It handles real-time code input, file uploads, and interactive style switching (Google vs. NumPy) with minimal latency.
- Export & Reporting Engine (ReportLab): A specialized library used to programmatically generate PDF reports. It translates the generated documentation into a professional, structured document with headers, tables, and formatted code blocks.
- UI Customization (HTML/CSS): The interface is enhanced with Custom CSS (Soft UI theme) to provide a modern "dark mode" aesthetic, improving the developer experience and visual clarity.
- Deployment Environment: Optimized for Google Colab and local environments, utilizing `pip` for dependency management of libraries like `transformers`, `torch`, and `gradio`.

6. Software Requirements

To ensure the high-performance execution of Doc-Genie v2.0, the following technical environment is required. This stack supports the advanced static analysis and the interactive web dashboard.

- Core Language: Python 3.8 or above. The system utilizes modern Python features like `ast.unparse` (available in Python 3.9+) and advanced type hinting for accurate signature extraction.
- Development Platform: Google Colab or a Local Jupyter Environment. This provides the necessary cloud-based compute power to handle library dependencies like `transformers` and `torch` efficiently.

- Dependency Management: pip Package Manager. The system relies on `pip` to install and update the core framework, including `gradio`, `reportlab`, and `ast-super-parser`.
- User Interface Access: Any modern Web Browser (Chrome, Firefox, or Edge). Since the frontend is powered by Gradio, it creates a local or public web URL that is accessible across all platforms without additional software.
- Network Connectivity: A stable Internet Connection. This is required for downloading the necessary Python libraries during the initial setup and for generating the shareable WhatsApp/Facebook links.

7. Hardware Requirements

Doc-Genie v2.0 is optimized for high efficiency and low resource consumption. Because it utilizes Static Code Analysis instead of real-time execution, it can run effectively on standard consumer-grade hardware. The following specifications are recommended for a smooth user experience:

- Processing Power: Intel Core i3 (or equivalent). The system requires basic multi-threading capabilities to handle the `ast` parsing and the Gradio web server simultaneously.
- System Memory (RAM): Minimum 2GB. While the core logic is lightweight, 2GB ensures sufficient overhead for the web browser and the Python environment to manage PDF generation via ReportLab.
- Disk Storage: 500MB Free Space. This provides ample room for the Python virtual environment, the required library dependencies, and the storage of exported PDF and TXT documentation reports.
- Visual Interface: 1024 x 768 Resolution. The Gradio dashboard is designed with a responsive "Soft UI" theme, ensuring that the code input area and the generated output remain clear and legible on standard displays.

8. Project Workflow

The development of Doc-Genie v2.0 followed a systematic five-stage engineering lifecycle. This ensured that the transition from a raw code input to a professional PDF documentation report was seamless and bug-free.

- Stage 1: Environment Setup & Dependency Integration: The initial phase involved configuring the Python 3.8+ environment within Google Colab. This included the automated installation of high-level libraries such as `Gradio` for the interface, `ReportLab` for document generation, and `ast-super-parser` for enhanced code traversal.

- Stage 2: Static Analysis Development (AST): This was the core engineering phase where the Abstract Syntax Tree (AST) logic was built. Instead of simple text matching, the workflow focused on "walking" through the code nodes (`ast.walk`) to identify parameters, return types, and operational logic like loops (`ast.For`) and math operations (`ast.BinOp`).
- Stage 3: Interactive Interface Design (Gradio): The workflow then shifted to the user experience. Using the Gradio framework, we developed a "Soft UI" dashboard that supports real-time code entry, file uploads, and a side-by-side view of the original code versus the newly documented version.
- Stage 4: Multi-Format Export Implementation: To make the tool useful in a professional setting, we integrated an export layer. This involved mapping the analyzed data into ReportLab templates to generate stylized PDFs and using `pathlib` to create downloadable TXT files.
- Stage 5: Testing, Validation & Deployment: The final stage involved rigorous testing with various code structures—from simple arithmetic to complex recursive functions. Once validated for accuracy and "Zero-Hallucination" documentation, the project was deployed as a shareable web application for collaborative use.

9. Features

Doc-Genie v2.0 is packed with developer-centric features that transform raw logic into professional-grade documentation. The system is designed for accuracy, flexibility, and ease of sharing.

- Autonomous Logic-Aware Docstring Synthesis: The system doesn't just "guess"; it performs a deep scan of your function's internal logic. By identifying loops, conditional branches, and mathematical operations, it generates a "plain-English" summary that accurately reflects what the code actually does.
- Dual-Standard Style Support: Flexibility is key in professional environments. Doc-Genie allows users to toggle between Google Style (the industry standard for general software) and NumPy Style (the gold standard for data science and scientific computing) with a single click.
- Deep Static Code Analysis (AST): Utilizing the Abstract Syntax Tree (AST), the tool extracts function signatures, parameter types, and return values without ever needing to execute the code. This ensures a 100% safe environment for analyzing untrusted or complex scripts.
- Integrated File Management & Uploads: Beyond simple copy-pasting, the system features a robust File Uploader. Developers can upload existing `.py` files directly into the dashboard for bulk analysis and documentation updates.
- Professional Multi-Format Exporting: To support offline reviews and project handovers, Doc-Genie features an export engine powered by ReportLab. Users can

download a beautifully formatted PDF report containing their code and its new documentation, or a lightweight TXT file for quick reference.
- Collaborative Sharing Module: Documentation is meant to be shared. The system includes built-in APIs to generate instant shareable links for WhatsApp and Facebook, allowing team members to quickly send documented snippets to one another.

10. Future Enhancements

While Doc-Genie v2.0 is currently a powerful utility for Python developers, the following roadmap outlines the planned evolution of the system to support a broader global developer community:

- Multi-Language Syntax Support: Expanding the core analysis engine to support languages beyond Python, such as Java, JavaScript, and C++. This would involve integrating specialized parsers to maintain the same high level of logic detection across different programming syntaxes.
- Seamless IDE Integration (VS Code Extension): To minimize context switching, the next phase involves developing a VS Code Extension. This would allow developers to highlight a function and generate a docstring directly within their code editor via a right-click menu or a keyboard shortcut.
- LLM-Powered Narrative Generation: While the current system uses deterministic AST logic, future versions will integrate Large Language Models (LLMs) to provide more "human-like" narrative summaries for highly complex algorithms, blending structural accuracy with natural language fluency.
- Automated GitHub Documentation Pipelines: Integration with GitHub Actions to automatically scan repositories and update documentation files (like `README.md` or `docs/`) whenever a developer pushes new code, ensuring that project documentation is never out of date.
- Integrated Code Quality & Complexity Metrics: Beyond documentation, the tool will be enhanced to provide Cyclomatic Complexity scores and "Clean Code" suggestions, helping developers not only document their functions but also optimize them for better performance and readability.

11. Conclusion

Doc-Genie v2.0 stands as a sophisticated, high-utility solution to one of the most persistent challenges in software engineering: the manual overhead of code documentation. By successfully bridging the gap between functional logic and human readability, the system proves that documentation does not have to be a secondary "chore" but can be a seamless part of the development lifecycle.

Through the strategic use of Static Code Analysis (AST) and a modern, responsive Gradio interface, Doc-Genie provides a "Zero-Hallucination" environment where documentation is generated based on the actual structural truth of the code. It empowers developers—from college students to industry professionals—to maintain clean, standardized, and well-documented codebases with minimal effort.

Ultimately, Doc-Genie is more than just a docstring generator; it is a productivity catalyst. It saves thousands of hours of developer time, reduces technical debt in legacy projects, and ensures that the next generation of Python developers can focus on what they do best: building the future, one function at a time.